

Spring Boot Documentation: Core Concepts, Configuration, and Best Practices

1. Introduction to Spring Boot

Spring Boot is a convention-over-configuration framework built on top of the Spring Framework. It simplifies the development of stand-alone, production-grade Java applications by providing opinionated defaults, embedded servers, and automatic dependency management. At its core, Spring Boot leverages **Dependency Injection** and the **IoC Container** to manage component lifecycles through the `ApplicationContext`.

Key characteristics include:

- Embedded servlet containers (Tomcat, Jetty, Undertow)
- Starter POMs for streamlined dependency resolution
- Externalized configuration via `application.properties` and `application.yml`
- Production-ready features out of the box

2. Core Concepts & Annotations

Spring Boot relies on a set of foundational annotations that drive auto-configuration and component scanning.

2.1 `@SpringBootApplication`

This composite annotation combines:

- `@EnableAutoConfiguration`: Triggers classpath-based auto-configuration
- `@ComponentScan`: Scans the current package and subpackages for `@Component`, `@Service`, `@Repository`, and `@Controller` beans
- `@Configuration`: Declares the class as a source of bean definitions

2.2 Bean Declaration & Injection

- `@Bean`: Declares a method return value as a managed bean in the `ApplicationContext`
- `@Autowired`: Injects dependencies by type. Can be used on fields, constructors, or setter methods
- `@Qualifier`: Resolves ambiguity when multiple beans match the same type

Example:

```
```java
```

```
@Configuration
```

```
public class DataConfig {
```

```

@Bean

public DataSource dataSource() {
 return new HikariDataSource();
}
}
...

```

### ## 3. Configuration & Properties Management

Spring Boot externalizes configuration through property sources loaded in a strict precedence order.

#### ### 3.1 `application.yml` vs `application.properties`

YAML provides hierarchical configuration, while properties use flat key-value pairs. Both support profile-specific overrides like `application-dev.yml`.

Example `application.yml`:

```

```yaml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/appdb
    username: root
    password: secret
  jpa:
    hibernate:
      ddl-auto: update
...

```

3.2 `@ConfigurationProperties`

Binds external configuration to strongly typed Java beans. Requires validation via `@Validated` and JSR-303 annotations.

```

```java
@ConfigurationProperties(prefix = "app.mail")

public class MailProperties {

```

```

 private String host;

 private int port;

 // getters & setters

}
...

```

## ## 4. REST API Development

Spring Boot simplifies RESTful web service creation using Spring Web MVC.

### ### 4.1 `@RestController` & `@RequestMapping`

`@RestController` combines `@Controller` and `@ResponseBody`, returning serialized JSON by default. `@RequestMapping` maps HTTP requests to handler methods.

```

```java

@RestController

@RequestMapping("/api/v1/users")

public class UserController {

    @GetMapping("/{id}")

    public ResponseEntity<UserDTO> getUser(@PathVariable Long id) { ... }

    @PostMapping

    public ResponseEntity<UserDTO> createUser(@Valid @RequestBody UserCreateRequest request) { ... }

}

...

```

4.2 Exception Handling

`@ControllerAdvice` and `@ExceptionHandler` centralize error responses:

```

```java

@ControllerAdvice

public class GlobalExceptionHandler {

 @ExceptionHandler(EntityNotFoundException.class)

 public ResponseEntity<ErrorResponse> handleNotFound(EntityNotFoundException ex) {

 return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new ErrorResponse(ex.getMessage()));

 }

}

```

```

 }

}

...

```

## ## 5. Data Access with Spring Data JPA

Spring Data JPA reduces boilerplate data access code by generating repository implementations at runtime.

### ### 5.1 Repository Interfaces

- `CrudRepository`: Basic CRUD operations
- `PagingAndSortingRepository`: Adds pagination and sorting
- `JpaRepository`: JPA-specific features like flush and batch delete

```
```java
```

```

public interface UserRepository extends JpaRepository<UserEntity, Long> {

    List<UserEntity> findByEmailContainingIgnoreCase(String email);

    Optional<UserEntity> findByUsername(String username);

}

...

```

5.2 Entity Mapping

Use `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, and relationship annotations (`@OneToOne`, `@ManyToOne`, `@JoinColumn`). Spring Boot automatically configures `EntityManagerFactory` and `PlatformTransactionManager` when `spring-boot-starter-data-jpa` is on the classpath.

6. Security & Authentication

Spring Security integrates seamlessly with Spring Boot via `spring-boot-starter-security`.

6.1 `@EnableWebSecurity` & `SecurityFilterChain`

Security configuration is defined through a `SecurityFilterChain` bean:

```
```java
```

```
@EnableWebSecurity
```

```
@Configuration
```

```
public class SecurityConfig {
```

@Bean

```
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
 return http
 .csrf(csrf -> csrf.disable())
 .authorizeHttpRequests(auth -> auth
 .requestMatchers("/api/public/**").permitAll()
 .anyRequest().authenticated()
)
 .httpBasic(Customizer.withDefaults())
 .build();
}
```

@Bean

```
public PasswordEncoder passwordEncoder() {
 return new BCryptPasswordEncoder();
}
```

```
}
```

```
...
```

### ### 6.2 Method-Level Security

Enable with `@EnableMethodSecurity`. Use `@PreAuthorize`, `@PostAuthorize`, `@Secured`, and `@RolesAllowed` for fine-grained access control.

## ## 7. Monitoring & Spring Boot Actuator

Spring Boot Actuator exposes operational endpoints for health, metrics, and diagnostics.

### ### 7.1 Core Endpoints

- `/actuator/health`: Application health status (UP, DOWN, OUT\_OF\_SERVICE)
- `/actuator/metrics`: JVM, HTTP, and custom metrics via Micrometer
- `/actuator/env`: Environment properties and active profiles
- `/actuator/info`: Custom application metadata

Enable in `application.yml`:

```

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,metrics,env,info
  endpoint:
    health:
      show-details: always
```

```

### ### 7.2 Custom Health Indicators

Implement `HealthIndicator` or extend `AbstractHealthIndicator` to report custom subsystem status.

## ## 8. Profiles & Environment Configuration

Spring Boot supports environment-specific configuration through profiles.

### ### 8.1 Activating Profiles

- `@ActiveProfiles("dev")` in tests
- `--spring.profiles.active=prod` CLI argument
- `SPRING_PROFILES_ACTIVE` environment variable

### ### 8.2 `@Profile` Annotation

Conditionally register beans:

```

```java
@Bean
@Profile("prod")
public DataSource prodDataSource() { ... }

@Bean
@Profile("dev,test")
public DataSource embeddedDataSource() { ... }
```

```

## ## 9. Testing Spring Boot Applications

Spring Boot provides first-class testing support via `@SpringBootTest` and test slices.

### ### 9.1 Integration Testing

```
```java
@SpringBootTest
@AutoConfigureMockMvc
class UserControllerIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void shouldReturnUser() throws Exception {
        mockMvc.perform(get("/api/v1/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.username").value("admin"));
    }
}
```
```

### ### 9.2 Test Slices

- `@WebMvcTest`: Tests controllers only
- `@DataJpaTest`: Tests JPA repositories with in-memory DB
- `@JsonTest`: Tests JSON serialization/deserialization

## ## 10. Deployment & Production Considerations

### ### 10.1 Packaging & Execution

- `mvn clean package` produces an executable JAR
- Run via `java -jar app.jar`
- Use `spring-boot-maven-plugin` for fat JAR creation

### ### 10.2 Graceful Shutdown & Health Checks

Enable graceful shutdown:

```
```yaml
```

```
server:
```

```
    shutdown: graceful
```

```
spring:
```

```
    lifecycle:
```

```
        timeout-per-shutdown-phase: 30s
```

```
```
```

### ### 10.3 Containerization

Dockerize with multi-stage builds. Expose port `8080` by default. Use `management.server.port` for separate actuator port if required.

---

## ## Glossary of Key Terms (Exact Match Reference)

| Term | Context |
|------|---------|
|------|---------|

|       |       |
|-------|-------|
| ----- | ----- |
|-------|-------|

|                          |                                                                                         |
|--------------------------|-----------------------------------------------------------------------------------------|
| `@SpringBootApplication` | Composite annotation enabling auto-configuration, component scanning, and configuration |
|--------------------------|-----------------------------------------------------------------------------------------|

|                            |                                             |
|----------------------------|---------------------------------------------|
| `@EnableAutoConfiguration` | Triggers classpath-driven bean registration |
|----------------------------|---------------------------------------------|

|                   |                                          |
|-------------------|------------------------------------------|
| `application.yml` | Hierarchical external configuration file |
|-------------------|------------------------------------------|

|                   |                                      |
|-------------------|--------------------------------------|
| `Spring Data JPA` | Abstraction layer over JPA providers |
|-------------------|--------------------------------------|

|                        |                                                      |
|------------------------|------------------------------------------------------|
| `Spring Boot Actuator` | Production-ready monitoring and management endpoints |
|------------------------|------------------------------------------------------|

|                       |                                                      |
|-----------------------|------------------------------------------------------|
| `SecurityFilterChain` | Core Spring Security request filtering configuration |
|-----------------------|------------------------------------------------------|

|            |                                                        |
|------------|--------------------------------------------------------|
| `@Profile` | Conditional bean registration based on active profiles |
|------------|--------------------------------------------------------|

|                   |                                       |
|-------------------|---------------------------------------|
| `@SpringBootTest` | Integration test bootstrap annotation |
|-------------------|---------------------------------------|

|          |                                    |
|----------|------------------------------------|
| `Tomcat` | Default embedded servlet container |
|----------|------------------------------------|

|                        |                                                 |
|------------------------|-------------------------------------------------|
| `Dependency Injection` | Core Spring IoC mechanism for wiring components |
|------------------------|-------------------------------------------------|